

Обработчики на завершение RPC-вызова

Встречаются ситуации, когда на завершение RPC-вызова необходимо выполнить действие, например сбросить [кэш метода бизнес-логики](#), создать запись на [сервисе истории](#) или отправить уведомление в [брокер синхронизации](#). Для решения подобных задач применяют API серверного фреймворка Wasaby, которое позволяет создавать обработчики (callback) и автоматически их выполнять на завершение RPC-вызова. Далее в статье рассмотрено доступное API, его ограничения, условия выполнения callback и примеры использования.

API C++

- [AddCurrentRpcCallFinishCallback](#) — создает анонимный/именованный (зависит от) callback, который будет вызван на завершение выполнения метода БЛ.
- [GetCurrentRpcCallFinishCallback](#) — выполняет поиск callback по имени и возвращает первый найденный callback.
- [GetOrAddCurrentRpcCallFinishCallback](#) — атомарно выполняет поиск callback по имени и возвращает первый найденный callback. Если callback отсутствует, тогда вызывается генератор для создания именованного callback.

Пример C++. Выполняется поиск callback с именем "MethodCacheCleaner". Для первого вызова в рамках текущего RPC-вызова callback не будет найден, а во всех остальных случаях — будет возвращен указатель на созданный callback. Далее приводим указатель базового класса INamedCallback в указатель дочернего класса CacheClearCallback.

```
1 CacheClearCallback* TryGetOrCreateCurrentRpcCallFinishCallback(  
2     std::shared_ptr< CacheDriver > const& driver )  
3 {  
4     return static_cast< CacheClearCallback* >(  
5         blcore::GetOrAddCurrentRpcCallFinishCallback( L"MethodCacheCleaner"_sv,  
6             [ &driver ]()  
7         )  
8     );  
9 }
```

Ниже показана реализация callback'a в классе CacheClearCallback. Он наследуется от базового для всех callback'ов класса INamedCallback. В классе CacheClearCallback происходит модификация состояния callback'a, указатель на который возвращается в предыдущем примере при вызове функции GetOrAddCurrentRpcCallFinishCallback. В классе перекрывается метод Execute, но при этом есть собственный экземплярный метод SynchronizedApply, который позволяет модифицировать (т.е. изменить) состояние mInvalidationRequest. В это состояние записываются те методы, для которых выполняется сброс кэша. В конечном итоге, когда метод БЛ завершился, в состоянии mInvalidationRequest будут присутствовать все уникальные методы, по которым необходимо сбросить кэш. При таком API происходит один вызов сервиса кэширования в отличие от использования API с анонимными callback.

```
1 class SBIS_HIDDEN_CLASS CacheClearCallback final : public  
2     INamedCallback  
3 {  
4 public:  
5     explicit CacheClearCallback( std::shared_ptr< CacheDriver > const&  
6         driver ) noexcept;  
7     void Execute() override;  
8     template< typename Callable >  
9     void SynchronizedApply( Callable&& func )  
10    {  
11        std::unique_lock< std::mutex > const lock{ mMutex };  
12        func( mInvalidationRequest );  
13    }
```

```

12
13 private:
14
15     std::shared_ptr< CacheDriver > mDriverPtr;
16     InvalidationRequest mInvalidationRequest;
17     mutable std::mutex mMutex;
18 };

```



Функции `GetCurrentRpcCallFinishCallback` и `AddCurrentRpcCallFinishCallback` (для именованного callback) считаются небезопасным API, потому что могут породить "гонки". Например, в одном из потоков выполняется поиск несуществующего callback по имени, а в параллельном потоке создается callback с таким же именем. Поэтому такие функции не имеют реализации в API Python.

API Python

- [AddCurrentRpcCallFinishCallback](#) — создает анонимный callback, который будет вызван на завершение выполнения метода БЛ.

```

1 class TestCase:
2
3     def my_unnamed_callback(self):
4         value = self.redis_client.Value("my_unnamed_callback")
5         value = int(value) + 1
6         self.redis_client.SetValue("my_unnamed_callback", str(value))
7
8     def test(self):
9         AddCurrentRpcCallFinishCallback(self.my_unnamed_callback)

```

- [GetOrAddCurrentRpcCallFinishCallback](#) — атомарно выполняет поиск callback по имени и возвращает первый найденный callback. Если callback отсутствует, тогда вызывается генератор для создания именованного callback.

```

1 class TestCase:
2
3     class MyINamedCallback:
4         def __init__(self, redis_client, name):
5             super().__init__();
6             self.redis_client = redis_client
7             self.inc = 1
8             self.name = name
9
10        def execute(self):
11            value = self.redis_client.Value(self.name)
12            value = int(value) + self.inc
13            self.redis_client.SetValue(self.name, str(value))
14
15        def set_inc(self, inc):
16            self.inc = inc
17
18        def get_inc(self):
19            return self.inc
20
21        def test(self):
22            GetOrAddCurrentRpcCallFinishCallback("my_callback_1", lambda
: TestCase.MyINamedCallback(self.redis_client, "my_callback_1"))

```



callback должны быть реализованы на Python. Т.е. из функции `GetOrAddCurrentRpcCallFinishCallback` нельзя запросить callback, реализованный на C++. Это породит ошибку.

Схемы работы с API

Анонимные callback'и

Функция: `AddCurrentRpcCallFinishCallback`

В этом случае анонимный callback ставится на исполнение по окончании выполнения метода БЛ. Сколько раз добавили callback, столько раз и вызывали.

Добавленный с помощью этой функции callback впоследствии нельзя как-либо найти и модифицировать. Из-за этого в некоторых задачах применение анонимных callback становится неэффективным.

Например, ранее анонимные callback'и использовались для решения задачи декларативного сброса кэша согласно [настройкам метода БЛ в Genie](#). После каждого вызова модифицирующего метода добавлялся анонимный callback на сброс кэша методов, результат работы которых потенциально меняет данный модифицирующий метод. Если модифицирующий метод вызывался в цикле, то таких callback (полностью идентичных) копилось по числу итераций цикла. В результате по окончании работы метода на сервис кэширования уходило N запросов, из которых полезную работу выполнял только первый. Для решения этой проблемы был введен более сложный API с [именованными callback'ами](#).

Именованные callback'и

Функция: `GetOrAddCurrentRpcCallFinishCallback`

В этом случае именованный callback создается единожды, а при поиске одноименного callback — возвращается указатель на существующий callback. В некоторых задачах это позволяет сэкономить ресурсы сервиса.



Внимание!

При передаче/захвате в callback объектов из текущей области видимости, как правило, необходимо делать глубокие (deep) копии данных объектов. Иначе в момент вызова callback данные объекты могут быть как-либо изменены, либо вовсе разрушены.

Зависимость от контекста выполнения метода БЛ

Функции API работают только в контексте выполнения метода БЛ, что определяется по наличию ключа `icsREQUEST_NUMBER` (см. [Get](#) (для [C++](#) или [Python](#)). Если контекст выполнения отсутствует, тогда в случае вызова функций возвращается ответ:

- `AddCurrentRpcCallFinishCallback` — `false` (для C++) или `False` (для Python);
- `GetCurrentRpcCallFinishCallback` и `GetOrAddCurrentRpcCallFinishCallback` — `nullptr` (для C++) или `None` (для Python).

При каких условиях вызываются callback?

1. Только в случае успешного завершения выполнения метода БЛ. Если выполнение метода прерывается исключением, тогда любые callback не вызываются.
2. До первого созданного исключения в ранее вызванных callback'ов.

Генерация исключений внутри callback

Если callback генерирует исключение, то оно регистрируется в логах приложения и обрабатывается серверным фреймворком, но вызов других callback не происходит. Поэтому не рекомендуется создавать исключения внутри callback'ов для RPC-вызовов.